Backpropagation Exercise (Answers)

Neural Computing

March 27, 2026

1 Single Perceptron

In this exercise, you will implement backpropagation by hand for the simplest possible neural network, a single perceptron with one input, one weight, one bias, and one output with sigmoid activation, on a binary classification task. This exercise will help you understand the fundamentals of gradient flows and weight update in the simplest case.

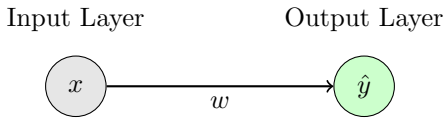

Figure 1: A single perceptron with one input x, one weight w, and one output $\hat{y}$. Note: The bias b is not shown in the diagram but is included in the calculations.

Data	Parameters	Functions
$x = 1$	$w = 0.8$	$\phi_{\text{output}}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$
$y = 0$	$b = -0.2$	$\mathcal{L}(\hat{y}, y) = \text{BCE}(\hat{y}, y) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$

Table 1: Input data, model parameters, and functions for the single perceptron.

Table 1 provides the values for our single perceptron model. Fill in the missing fields. Since we are working with a binary classification task, a natural choice for the activation for the output layer is sigmoid activation. Why is this the case? For the loss function, we use Binary Cross-Entropy loss (BCE). Why is this suitable for binary classification tasks?

Then, let us work through how to implement backpropagation step-by-step.

1. Forward pass
   (a) Calculate the pre-activation value $z = w \cdot x + b$:

   This is just the weighted sum of inputs plus bias. In this case, we only have one input.

   $$z = w \cdot x + b = 0.8 \cdot 1 + (-0.2) = 0.6$$

   (b) Calculate the post-activation value/output $\hat{y} = \sigma(z)$:

   We apply non-linearity to the pre-activation value.

   $$\hat{y} = \sigma(0.6) = \frac{1}{1 + e^{-0.6}} \approx 0.6457$$

   (c) Calculate the loss $L = \mathcal{L}(\hat{y}, y) = \text{BCE}(\hat{y}, y)$:

   We check how good/bad our prediction is. Lower values mean better performance.

   $$\begin{aligned} L &= -[0 \cdot \log(0.6457) + 1 \cdot \log(1 - 0.6457)] \\ &= -\log(0.3543) \\ &\approx 1.0377 \end{aligned}$$

2. Backward Pass
   (a) Compute the gradient of the loss with respect to the output $\left(\frac{\partial L}{\partial \hat{y}}\right)$:

   This gradient shows how sensitive the loss is to small changes in our prediction.

   $$\begin{aligned} \frac{\partial L}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})} \\ &= \frac{1}{0.3543} \\ &\approx 2.8225 \end{aligned}$$

   (b) Compute the gradient of the output with respect to the pre-activation value $\left(\frac{\partial \hat{y}}{\partial z}\right)$:

   This is the derivative of the sigmoid function, showing how the output changes when the pre-activation changes.

   $$\begin{aligned} \frac{\partial \hat{y}}{\partial z} &= \sigma(z)(1 - \sigma(z)) \\ &= 0.6457 \cdot 0.3543 \\ &\approx 0.2288 \end{aligned}$$

(c) Compute the gradient of the loss with respect to the pre-activation value by applying the chain rule $\left(\frac{\partial L}{\partial z}\right)$:

This is the central magic of backpropagation. The chain rule allows us to connect the loss to the pre-activation, showing how changes in z affect the loss.

$$\begin{aligned}\frac{\partial L}{\partial z} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \\ &= 2.8225 \cdot 0.2288 \\ &\approx 0.6458\end{aligned}$$

(d) Compute the gradient of the loss with respect to the weight $\left(\frac{\partial L}{\partial w}\right)$:

This gradient tells us how changing the weight, a critical value for learning, affects the loss.

$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial w} \\ &= 0.6458 \cdot 1 \\ &\approx 0.6458\end{aligned}$$

(e) Compute the gradient of the loss with respect to the bias $\left(\frac{\partial L}{\partial b}\right)$:

Similarly, this shows how the bias impacts the loss; note that the input for bias is always 1.

$$\begin{aligned}\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial b} \\ &= 0.6458 \cdot 1 \\ &\approx 0.6458\end{aligned}$$

3. Update Parameters

   (a) Update the weight using gradient descent:

   Now that we have the gradient of the loss with respect to the weight, we can use it to adjust the weight in the opposite direction of the gradient to reduce the loss. However, we don't want to update too quickly, since that could cause us to overshoot the minimum, leading to unstable training. So, we use a learning rate η, set to 0.1 this time, to control the step size of our parameter updates.

$$\begin{aligned} w' &= w - \eta \cdot \frac{\partial L}{\partial w} \\ &= 0.8 - 0.1 \cdot 0.6458 \\ &\approx 0.7354 \end{aligned}$$

   (b) Update the bias using gradient descent:

   Similarly, we update the bias to move toward lower loss values.

$$\begin{aligned} b' &= b - \eta \cdot \frac{\partial L}{\partial b} \\ &= -0.2 - 0.1 \cdot 0.6458 \\ &\approx -0.2646 \end{aligned}$$

4. Second forward pass

   Using the updated weights, perform a second forward pass and calculate the new loss. Is the prediction better?

   Using updated parameters:

   - $\hat{y}' = \sigma(0.7354 \cdot 1 + (-0.2646)) = 0.6156$
   - $L' = 0.9562$

   Since the new loss is less than the original, the prediction is better.

2 Multi-Layer Perceptron

Building on Exercise 1, this exercise applies backpropagation to a larger network with 2 inputs, 2 hidden neurons, and 1 output neuron.

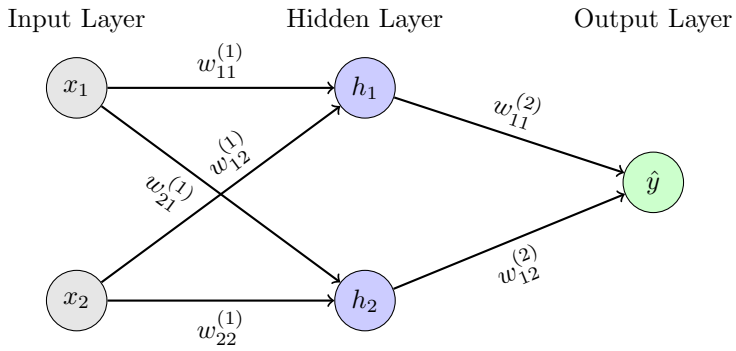

Figure 2: A MLP with two inputs, two hidden neurons, and one output. Note: The biases $(b^{(d)}, d \in \{1, 2\})$ are not shown in the diagram but the terms are included in the calculations.

Data	Parameters	Functions and Hyperparameters
$X_1 = \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix}$	$W^{(1)} = \begin{bmatrix} 0.2 & 0.4 \\ 0.3 & -0.1 \end{bmatrix}$	$\phi_{\text{hidden}}(z) = \text{ReLU}(z) = \max(0, z)$
$X_2 = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}$	$W^{(2)} = \begin{bmatrix} 0.5 & 0.6 \end{bmatrix}$	$\phi_{\text{output}}(z) = z$
$y_1 = 3.5$	$b^{(1)} = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$	$\mathcal{L}(\hat{y}, y) = \text{SEL}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$
$y_2 = 2.0$	$b^{(2)} = 0.1$	$\eta = 0.1$

Table 2: Input data, model parameters, functions, and hyperparameters for the MLP.

This time, let us work with a regression task. You will work with two data points and trace the complete process of a mini-batch gradient descent update. This exercise will help you understand how gradients flow backward through, and how weights are updated, in a larger network with hidden layer(s).

First, fill in the missing fields in Table 2. For the activation of the hidden layer, we use ReLU activation. What should the output-layer activation function be for a regression task? For the loss function, we use squared error loss (SEL). Why?

Then, for each data point X_1 and X_2, perform the following steps:

1. Forward Pass
   (a) Calculate the hidden layer pre-activation $z^{(1)}$:

   For X_1:

   $$\begin{bmatrix} 1.1 \\ 0.3 \end{bmatrix}$$

   For X_2:

   $$\begin{bmatrix} 0.6 \\ 0.25 \end{bmatrix}$$

   (b) Calculate the hidden layer h:

   For X_1:

   $$\begin{bmatrix} 1.1 \\ 0.3 \end{bmatrix}$$

   For X_2:

   $$\begin{bmatrix} 0.6 \\ 0.25 \end{bmatrix}$$

   (c) Calculate the output layer pre-activation $z^{(2)}$:

   For X_1: 0.83, For X_2: 0.55

   (d) Calculate the output $\hat{y}$:

   For X_1: 0.83, For X_2: 0.55

   (e) Calculate the loss L:

   For X_1: 3.5645, For X_2: 1.0513

2. Backward Pass
   (a) Calculate the gradient of the loss w.r.t. the output ($\nabla_{\hat{y}} L$):

   For X_1: -2.67, For X_2: -1.45

   (b) Calculate the gradient of the loss w.r.t. the output pre-activation ($\nabla_{z^{(2)}} L$):

   For X_1: -2.67, For X_2: -1.45

   (c) Calculate the gradient of the loss w.r.t. $W^{(2)}$ ($\nabla_{W^{(2)}} L$):

   For X_1:

   $$\begin{bmatrix} -2.937 \\ -0.801 \end{bmatrix}$$

   For X_2:

   $$\begin{bmatrix} -0.87 \\ -0.3625 \end{bmatrix}$$

(d) Calculate the gradient of the loss w.r.t. $b^{(2)}$ ($\nabla_{b^{(2)}} L$):

For X_1: -2.67, For X_2: -1.45

(e) Calculate the gradient of the loss w.r.t. the hidden layer ($\nabla_h L$):

For X_1:

$$\begin{bmatrix} -1.335 \\ -1.602 \end{bmatrix}$$

For X_2:

$$\begin{bmatrix} -0.725 \\ -0.87 \end{bmatrix}$$

(f) Calculate the gradient of the loss w.r.t. the hidden layer pre-activation ($\nabla_{z^{(1)}} L$):

For X_1:

$$\begin{bmatrix} -1.335 \\ -1.602 \end{bmatrix}$$

For X_2:

$$\begin{bmatrix} -0.725 \\ -0.87 \end{bmatrix}$$

(g) Calculate the gradient of the loss w.r.t. $W^{(1)}$ ($\nabla_{W^{(1)}} L$):

For X_1:

$$\begin{bmatrix} -1.335 & -2.67 \\ -1.602 & -3.204 \end{bmatrix}$$

For X_2:

$$\begin{bmatrix} -0.3625 & -0.725 \\ -0.435 & -0.87 \end{bmatrix}$$

(h) Calculate the gradient of the loss w.r.t. $b^{(1)}$ ($\nabla_{b^{(1)}} L$):

For X_1:

$$\begin{bmatrix} -1.335 \\ -1.602 \end{bmatrix}$$

For X_2:

$$\begin{bmatrix} -0.725 \\ -0.87 \end{bmatrix}$$

3. Weight update

   (a) For all parameters, calculate average gradients across both data points

      i. Calculate the average gradient of $W^{(1)}$:

$$\begin{bmatrix} -0.849 & -1.698 \\ -1.019 & -2.037 \end{bmatrix}$$

      ii. Calculate the average gradient of $b^{(1)}$:

$$\begin{bmatrix} -1.03 \\ -1.236 \end{bmatrix}$$

      iii. Calculate the average gradient of $W^{(2)}$:

$$\begin{bmatrix} -1.904 \\ -0.582 \end{bmatrix}$$

      iv. Calculate the average gradient of $b^{(2)}$:

      -2.06

   (b) Update the weights and biases using the learning rate $\eta = 0.1$

      i. Update $W^{(1)}$:

$$\begin{bmatrix} 0.285 & 0.570 \\ 0.402 & 0.104 \end{bmatrix}$$

      ii. Update $b^{(1)}$:

$$\begin{bmatrix} 0.203 \\ 0.324 \end{bmatrix}$$

      iii. Update $W^{(2)}$:

$$\begin{bmatrix} 0.690 \\ 0.658 \end{bmatrix}$$

      iv. Update $b^{(2)}$:

      0.306

4. Second forward pass

   Using the updated weights, perform a second forward pass for both data points and calculate the new loss. Is the prediction better?

$$\hat{y}_1 \approx 2.044, \hat{y}_2 \approx 1.352, L_1 \approx 1.060, L_2 \approx 0.210$$